

C++ (Quant Dev) – The Project

Mini Exchange Simulator + Disruptor-Lite

Année : 2026

Nom : _____

0. Règles générales

1. Le livrable est un **repo GitHub** contenant uniquement du code source (.h/.cpp) et la config de build (CMakeLists.txt).
2. Le projet doit compiler en **C++20** (C++17 toléré si vous expliquez pourquoi dans le README).
3. Build **CMake** obligatoire. Aucune dépendance payante.
4. Dépendances autorisées (0€) : Boost.Asio (réseau), Catch2 (tests), nlohmann/json (JSON), fmt/spdlog (logs).
5. **Interdit** : ajouter des **membres publics** “gratuits” dans les classes demandées. Attributs **privés** + getters si besoin.
6. **Concurrence** : l'**OrderBook** ne doit être modifié **que** par le thread **MatchingEngine**.
7. Deux diagrammes sont requis dans le rendu (README ou PDF) :
 - diagramme d'architecture (composants + flèches),
 - diagramme de séquence (un scénario d'exécution).

1. Contexte (idée générale)

On veut simuler une mini-bourse en **limit order book** :

- Les clients envoient des ordres (**Limit**, **Market**, **Cancel**).
- Le serveur maintient un carnet : côté **bid** (achats) et côté **ask** (ventes).
- Le **matching** suit **price-time priority** :
 - meilleur prix d'abord (*best bid* le plus haut, *best ask* le plus bas),
 - à prix égal, **FIFO** (ordre d'arrivée).
- Le serveur renvoie des événements : **Ack**, **Reject**, **Fill**.

Le projet comporte aussi un bus interne **Disruptor-Lite** : un **ring buffer SPSC** pour transférer les messages entre threads. SPSC = **S**ingle **P**roducer **S**ingle **C**onsumer (un producteur, un consommateur).

Architecture cible (à mettre dans votre diagramme) :

Client ↔ TCP → Decode → IN Bus → Matching → OUT Bus → Encode → TCP ↔ Client.

Part I

Order Book & Matching Engine (single-thread)

1. Spécification

On définit :

- **Side** : BUY ou SELL.
- **Symbol** : chaîne (ex : "BTC", "AAPL").

- **Price** et **Qty** : strictement positifs pour les ordres concernés.

Types de commandes (entrée) :

- **Limit** : (clientId, orderId, symbol, side, price, qty)
- **Market** : (clientId, orderId, symbol, side, qty)
- **Cancel** : (clientId, orderId, symbol)

Types d'événements (sortie) :

- **Ack** : commande acceptée
- **Reject** : commande refusée + raison
- **Fill** : exécution (matchId, price, qty) (partielle ou totale)

Exigences de structure (OrderBook) :

- Le carnet est organisé en **niveaux de prix** (price levels).
- À chaque niveau de prix, les ordres sont stockés en **FIFO** (time priority).
- Un index interne **orderId** → **référence** est **obligatoire** pour implémenter **Cancel** sans parcourir tout le carnet.
- Les meilleurs prix (*best bid* / *best ask*) doivent être accessibles **sans scan complet** du carnet.

2. Implementation (classes imposées)

1. Implémenter **enum class Side** avec deux valeurs : **Buy**, **Sell**.
2. Implémenter **enum class CommandType** : **Limit**, **Market**, **Cancel**.
3. Implémenter **enum class EventType** : **Ack**, **Reject**, **Fill**.
4. Implémenter **enum class RejectReason** contenant au minimum : **INVALID_QTY**, **INVALID_PRICE**, **UNKNOWN_SYMBOL**, **UNKNOWN_ORDER_ID**, **NOT_OWNER**, **INTERNAL_ERROR**.
5. Implémenter la classe abstraite **Command** :
 - membres privés : **_clientId**, **_orderId**, **_symbol**
 - getters : **getClientId()**, **getOrderId()**, **getSymbol()**
 - constructeur initialisant ces champs
 - destructeur virtuel
 - méthode pure virtuelle : **CommandType getType() const**
6. Dériver **Command** en trois classes :
 - **LimitCommand** :
 - membres privés : **_side**, **_price**, **_qty**
 - getters : **getSide()**, **getPrice()**, **getQty()**
 - le constructeur doit vérifier : **price > 0** et **qty > 0** sinon erreur (à votre choix : exception ou **Reject** en amont)
 - **MarketCommand** :
 - membres privés : **_side**, **_qty**
 - le constructeur doit vérifier : **qty > 0**
 - **CancelCommand** :
 - aucun membre supplémentaire obligatoire
7. Implémenter la classe abstraite **Event** :
 - membres privés : **_clientId**, **_orderId**
 - getters : **getClientId()**, **getOrderId()**

- constructeur initialisant ces champs
 - destructeur virtuel
 - méthode pure virtuelle : `EventType getType() const`
8. Dériver **Event** en trois classes :
- **AckEvent** : pas d'attribut supplémentaire obligatoire
 - **RejectEvent** :
 - membre privé : `_reason` (type `RejectReason`)
 - getter : `getReason()`
 - **FillEvent** :
 - membres privés : `_matchId`, `_price`, `_qty`
 - getters correspondants
9. Implémenter une classe **Order** (objet stocké dans le book) :
- membres privés minimum : `_clientId`, `_orderId`, `_side`, `_price`, `_qtyRemaining`
 - getters + une méthode pour décrémenter la quantité restante lors d'un fill
10. Implémenter la classe **OrderBook** :
- contient deux côtés : bids et asks
 - doit permettre :
 - insérer un **Order** (limit) au bon niveau de prix
 - accéder au meilleur bid / meilleur ask
 - retirer les ordres totalement exécutés
 - gérer **Cancel(orderId)** (avec contrôle du `clientId`)
 - **Exigence** : FIFO à prix égal (vous devez donc stocker les ordres à un prix donné dans une structure FIFO).
 - **Exigence** : **Cancel** ne doit pas être implémenté en “parcourant tout le book” (il faut un index interne par `orderId`).
11. Implémenter la classe **MatchingEngine** :
- membres privés : `_book` (`OrderBook`), `_nextMatchId`
 - méthode publique : `process(const Command&)` qui renvoie une collection d'`Event`
 - règles :
 - **Limit** : si crossing, exécuter immédiatement contre le côté opposé puis (si reste) insérer le reste dans le book
 - **Market** : exécuter tant qu'il y a de la liquidité, puis s'arrêter (le reste est abandonné)
 - **Cancel** : si inconnu $\Rightarrow$ Reject ; si `clientId` ne correspond pas $\Rightarrow$ Reject (`NOT_OWNER`)

3. Tests unitaires (obligatoires)

Écrire au minimum 8 tests couvrant :

1. insertion d'une limit sans crossing $\Rightarrow$ book mis à jour
2. limit crossing $\Rightarrow$ fill total
3. limit crossing $\Rightarrow$ fill partiel + reste au book
4. market qui balaye plusieurs niveaux
5. FIFO à prix égal (ordre A avant ordre B)

6. cancel d'un ordre existant
7. cancel d'un ordre déjà rempli $\Rightarrow$ reject
8. cancel d'un orderId inconnu $\Rightarrow$ reject

4. Points difficiles (à lire avant de coder)

- **FIFO** : si votre structure de niveau de prix n'est pas une file, vous casserez la priorité temps.
- **Cancel** : sans index par `orderId`, vous aurez une implémentation lente et fragile.

Part II

Protocole + TCP Server/Client

1. Contexte (mix)

TCP est un flux d'octets : un `read()` ne correspond pas à "un message". On impose donc un **framing length-prefix** : d'abord la taille, puis le payload.

2. Spécification du framing (obligatoire)

- Message = `uint32_t len` (network byte order) suivi de `len` octets (payload).
- Le payload doit représenter une **Command** (client $\rightarrow$ serveur) ou un **Event** (serveur $\rightarrow$ client).

3. Implementation (classes imposées)

1. Implémenter une classe **Framer** :

- méthode `frame(payloadBytes) -> framedBytes`
- méthode `consume(buffer) -> (0 ou plusieurs messages complets)`
- **Exigence** : `consume` doit gérer les lectures partielles (payload incomplet) et conserver l'état interne.

2. Implémenter une classe **MessageCodec** :

- `encodeCommand(const Command&) -> payloadBytes`
- `decodeCommand(payloadBytes) -> Command`
- `encodeEvent(const Event&) -> payloadBytes`
- `decodeEvent(payloadBytes) -> Event` (si nécessaire côté client)
- Le format est libre (JSON autorisé) mais doit supporter tous les champs requis en Part I.

3. Implémenter une classe **ExchangeServer** :

- accepte plusieurs clients TCP
- pour chaque client : lit, reconstitue les messages via **Framer**, décode via **MessageCodec**
- envoie la commande vers le pipeline interne (Part III)
- renvoie les événements au client concerné

4. Implémenter une classe **ExchangeClient** (CLI) :

- se connecte au serveur
- lit des commandes texte (stdin) et envoie les commandes au serveur
- affiche les événements reçus

4. Point difficile

- Reconstituer proprement les messages : buffer + len + payload, sans perdre de données.
- Gérer les déconnexions sans crash (serveur robuste).

Part III

Disruptor-Lite (Ring Buffer SPSC) + Pipeline multi-thread

1. Contexte (mix)

Un ring buffer SPSC est un tableau circulaire partagé entre :

- un producteur (qui écrit à l'index *tail*),
- un consommateur (qui lit à l'index *head*).

Le producteur ne doit jamais écraser un slot non consommé (buffer plein). Le consommateur ne doit jamais lire un slot non écrit (buffer vide).

2. Implementation (classe imposée)

1. Implémenter une classe template **RingBuffer<T>** :

- membres privés :
 - un buffer pré-alloué de capacité fixe
 - deux compteurs atomiques `_head` et `_tail` (avec padding/alignement pour éviter le false sharing)
 - la capacité (puissance de 2 recommandée)
- méthodes publiques :
 - `bool tryPush(const T&)`
 - `bool tryPop(T&)`
 - `size_t capacity() const`
- **Exigence** : aucune allocation mémoire dans `tryPush/tryPop`.
- **Exigence** : utiliser `std::atomic` et une discipline de visibilité mémoire correcte (acquire/release).

2. Créer deux instances :

- **IN_BUS** : transporte les **Command** du thread réseau vers le thread matching,
- **OUT_BUS** : transporte les **Event** du thread matching vers le thread réseau.

3. Pipeline multi-thread (obligatoire) :

- Thread réseau : reçoit **Command** → push dans **IN_BUS**
- Thread matching : pop **IN_BUS** → **MatchingEngine** → push **OUT_BUS**
- Thread réseau : pop **OUT_BUS** → renvoie au client

4. Politique buffer plein (à fixer et documenter) :

- Si **IN_BUS** est plein, vous devez choisir une politique unique (ex : rejet immédiat, ou attente active).
- Cette politique doit être écrite dans le README et testée au moins une fois.

3. Points difficiles (à lire avant de coder)

- Si votre ordering atomique est faux, vous aurez des bugs **rares** et difficiles à reproduire.
- Si votre padding est absent, vous pouvez observer des performances instables (false sharing).

Part IV

Rendu, diagrammes, démo

1. Diagrammes (obligatoires)

1. **Architecture** : client, TCP, framer, codec, IN_BUS, matching, book, OUT_BUS.
2. **Séquence** : scénario minimal :
 - client pose une limit SELL,
 - client pose une limit BUY crossing,
 - serveur renvoie Ack + Fill(s),
 - book mis à jour.

2. Démo minimale (obligatoire dans le README)

Le README doit contenir un scénario reproductible :

- commande pour compiler
- commande pour lancer le serveur
- commandes pour lancer 1 ou 2 clients
- séquence d'ordres + résultat attendu (ACK/FILL/REJECT)

3. Tests d'intégration (obligatoire)

Au moins un test (ou script) qui :

- lance le serveur,
- envoie une séquence d'ordres,
- vérifie la présence et l'ordre des événements attendus.